

PhoneBook — Practice 2

PP2 · Programming Principles 2 | PostgreSQL Functions & Procedures

- i Prerequisites: phonebook table must exist from Practice 1. Run all SQL blocks inside psql shell. Python snippets show how to call each function/procedure from your phonebook.py.

Search function — return all records matching a pattern

A scalar function that searches first_name or phone using ILIKE. Returns a table of matching rows.

A Create the function in psql

This function accepts a search pattern and returns all matching rows as a table:

psql shell

```
CREATE OR REPLACE FUNCTION search_phonebook(pattern VARCHAR)
RETURNS TABLE(id INT, first_name VARCHAR, phone VARCHAR)
LANGUAGE plpgsql
AS $$
BEGIN
RETURN QUERY
SELECT p.id, p.first_name, p.phone
FROM phonebook p
WHERE p.first_name ILIKE '%' || pattern || '%'
OR p.phone ILIKE '%' || pattern || '%'
ORDER BY p.first_name;
END;
$$;
```

B Test it in psql

psql shell

```
-- Search by part of name:
SELECT * FROM search_phonebook('Ali');

-- Search by phone prefix:
SELECT * FROM search_phonebook('+7700');

-- Search anything containing '123':
SELECT * FROM search_phonebook('123');
```

C Call it from Python (phonebook.py)

phonebook.py

```
def search_contacts(pattern):
    cur.execute(
        "SELECT * FROM search_phonebook(%s);",
        (pattern,)
    )
    rows = cur.fetchall()
    for r in rows:
        print(f" {r[0]:<4} {r[1]:<20} {r[2]}")

search_contacts('Ali') # finds Alice, Alikhan...
search_contacts('+7700') # finds by phone prefix
```

i ILIKE is case-insensitive. The || operator concatenates strings in SQL. '%' || pattern || '%' means 'contains pattern anywhere'.

Upsert procedure — insert new user, or update phone if exists

A procedure that inserts a new contact. If the name already exists, it updates their phone instead. This pattern is called UPSERT.

A Create the procedure

psql shell

```
CREATE OR REPLACE PROCEDURE upsert_contact(  
  p_name VARCHAR,  
  p_phone VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  -- Try to insert; if phone already exists, update name  
  INSERT INTO phonebook (first_name, phone)  
  VALUES (p_name, p_phone)  
  ON CONFLICT (phone)  
  DO UPDATE SET first_name = EXCLUDED.first_name;  
  
  RAISE NOTICE 'Upserted: % - %', p_name, p_phone;  
END;  
$$;
```

B Test it in psql

psql shell

```
-- Insert new contact:  
CALL upsert_contact('Alice', '+77001112233');  
  
-- Same phone, different name -> updates name:  
CALL upsert_contact('Aliya', '+77001112233');  
  
-- Verify:  
SELECT * FROM phonebook WHERE phone = '+77001112233';
```

C Call it from Python

phonebook.py

```
def upsert_contact(name, phone):  
    cur.execute(  
        "CALL upsert_contact(%, %s);",  
        (name, phone)  
    )  
    conn.commit()  
    print(f"[OK] Upserted: {name} - {phone}")  
  
upsert_contact('Alice', '+77001112233') # insert  
upsert_contact('Aliya', '+77001112233') # update
```

O EXCLUDED.first_name refers to the value that was attempted to be inserted but conflicted. ON CONFLICT
K (phone) DO UPDATE means: if phone already exists, update instead of failing.

Bulk insert procedure — validate phones, return bad data

A procedure that accepts a list of names and phones, loops through them, validates each phone, inserts valid ones, and collects invalid entries to return.

A Create the procedure with loop and validation

psql shell

```
CREATE OR REPLACE PROCEDURE insert_many_users(
  p_names VARCHAR[],
  p_phones VARCHAR[]
)
LANGUAGE plpgsql
AS $$
DECLARE
  i INT;
  v_name VARCHAR;
  v_phone VARCHAR;
BEGIN
  -- Loop through each pair by index
  FOR i IN 1 .. array_length(p_names, 1) LOOP
    v_name := p_names[i];
    v_phone := p_phones[i];

    -- Validate: phone must start with + and be 12 chars
    IF v_phone NOT LIKE '+%'
    OR LENGTH(v_phone) < 11
    OR LENGTH(v_phone) > 13 THEN

      RAISE NOTICE 'INVALID: % / %', v_name, v_phone;
    ELSE
      INSERT INTO phonebook (first_name, phone)
      VALUES (v_name, v_phone)
      ON CONFLICT (phone) DO NOTHING;
      RAISE NOTICE 'INSERTED: % / %', v_name, v_phone;
    END IF;
  END LOOP;
END;
$$;
```

B Test it in psql

psql shell

```
CALL insert_many_users(
  ARRAY['Alice', 'Bob', 'Bad1', 'Zara'],
  ARRAY['+77001112233', '+77009998877',
  '12345', '+77712345678']
);

-- Output:
-- NOTICE: INSERTED: Alice / +77001112233
-- NOTICE: INSERTED: Bob / +77009998877
-- NOTICE: INVALID: Bad1 / 12345
-- NOTICE: INSERTED: Zara / +77712345678
```

C Call it from Python

phonebook.py

```
def insert_many(contacts):
    # contacts = [('Alice', '+77001112233'), ('Bob', '12345')]
    names = [c[0] for c in contacts]
    phones = [c[1] for c in contacts]
    cur.execute(
        "CALL insert_many_users(%s::VARCHAR[], %s::VARCHAR[]);",
        (names, phones)
    )
    conn.commit()
    print("[OK] Bulk insert done. Check NOTICE output for invalids.")

insert_many([
    ('Alice', '+77001112233'),
    ('Bob', '12345'), # invalid phone
    ('Zara', '+77712345678'),
])
```

! Phone validation rule here: must start with + and be 11-13 characters long. You can adjust the rule — for example, require exactly 12 chars for Kazakh numbers (+7XXXXXXXXXX).

Pagination function — query data with LIMIT and OFFSET

A function that returns one page of contacts at a time. Pass the page number and page size — it calculates LIMIT and OFFSET automatically.

A Create the pagination function

psql shell

```
CREATE OR REPLACE FUNCTION get_phonebook_page(
  p_page INT DEFAULT 1,
  p_page_size INT DEFAULT 5
)
RETURNS TABLE(id INT, first_name VARCHAR, phone VARCHAR)
LANGUAGE plpgsql
AS $$
DECLARE
  v_offset INT;
BEGIN
  -- Calculate offset from page number
  v_offset := (p_page - 1) * p_page_size;

  RETURN QUERY
  SELECT p.id, p.first_name, p.phone
  FROM phonebook p
  ORDER BY p.id
  LIMIT p_page_size
  OFFSET v_offset;
END;
$$;
```

B Test it in psql

psql shell

```
-- Page 1 (contacts 1-5):
SELECT * FROM get_phonebook_page(1, 5);

-- Page 2 (contacts 6-10):
SELECT * FROM get_phonebook_page(2, 5);

-- Page 1 with 3 per page:
SELECT * FROM get_phonebook_page(1, 3);

-- Using defaults (page=1, size=5):
SELECT * FROM get_phonebook_page();
```

C How LIMIT and OFFSET are calculated

Logic

```
-- Formula:
-- OFFSET = (page - 1) * page_size

-- Page 1, size 5: OFFSET = (1-1)*5 = 0 -> rows 1..5
-- Page 2, size 5: OFFSET = (2-1)*5 = 5 -> rows 6..10
-- Page 3, size 5: OFFSET = (3-1)*5 = 10 -> rows 11..15
-- Page 1, size 3: OFFSET = (1-1)*3 = 0 -> rows 1..3
-- Page 2, size 3: OFFSET = (2-1)*3 = 3 -> rows 4..6
```

D Call it from Python with navigation

phonebook.py

```
def show_page(page=1, page_size=5):
    cur.execute(
        "SELECT * FROM get_phonebook_page(%s, %s);",
        (page, page_size)
    )
    rows = cur.fetchall()
    print(f"\n--- Page {page} ---")
    if not rows:
        print(" (no more contacts)")
    for r in rows:
        print(f" {r[0]:<4} {r[1]:<20} {r[2]}")

show_page(1, 5) # first page
show_page(2, 5) # second page
```

- i DEFAULT values in the function signature mean the parameter is optional. Always use ORDER BY when paginating — without it, the database may return rows in a different order each time.

Delete procedure — remove contact by username or phone

A procedure that deletes a contact. You can pass either a name or a phone number — whichever you provide, it finds and deletes the matching row.

A Create the delete procedure

psql shell

```
CREATE OR REPLACE PROCEDURE delete_contact(  
  p_name VARCHAR DEFAULT NULL,  
  p_phone VARCHAR DEFAULT NULL  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
  v_deleted INT;  
BEGIN  
  IF p_name IS NOT NULL THEN  
    DELETE FROM phonebook  
    WHERE first_name ILIKE p_name;  
    GET DIAGNOSTICS v_deleted = ROW_COUNT;  
    RAISE NOTICE 'Deleted % row(s) by name: %',  
      v_deleted, p_name;  
  
    ELSIF p_phone IS NOT NULL THEN  
    DELETE FROM phonebook  
    WHERE phone = p_phone;  
    GET DIAGNOSTICS v_deleted = ROW_COUNT;  
    RAISE NOTICE 'Deleted % row(s) by phone: %',  
      v_deleted, p_phone;  
  
    ELSE  
    RAISE EXCEPTION 'Provide p_name or p_phone!';  
  END IF;  
END;  
$;
```

B Test it in psql

psql shell

```
-- Delete by name:  
CALL delete_contact(p_name => 'Alice');  
  
-- Delete by phone:  
CALL delete_contact(p_phone => '+77009998877');  
  
-- This raises an exception (no argument given):  
CALL delete_contact();
```

C Call it from Python

phonebook.py

```
def delete_contact(name=None, phone=None):
    if name:
        cur.execute(
            "CALL delete_contact(p_name => %s);", (name,)
        )
    elif phone:
        cur.execute(
            "CALL delete_contact(p_phone => %s);", (phone,)
        )
    else:
        print("[ERROR] Provide name or phone.")
        return
    conn.commit()
    print("[OK] Delete done.")

delete_contact(name='Alice')
delete_contact(phone='+77009998877')
```

- O** GET DIAGNOSTICS v_deleted = ROW_COUNT gives you how many rows were deleted. RAISE
- K** EXCEPTION stops execution and rolls back — good for catching programmer mistakes like calling delete with no arguments.

Task	Type	Call from psql	Call from Python
Search by name/phone	FUNCTION	SELECT * FROM search_phonebook('Ali');	cur.execute("SELECT * FROM search_phonebook(%s);", (q,))
Insert or update	PROCEDURE	CALL upsert_contact('Ali', '+7700...');	cur.execute("CALL upsert_contact(%s,%s);", (n,p))
Bulk insert + validate	PROCEDURE	CALL insert_many_users(ARRAY[...], ARRAY[...]);	cur.execute("CALL insert_many_us ers(%s::VARCHAR[],%s::VARCHAR[]) ;", (names,phones))
Paginated query	FUNCTION	SELECT * FROM get_phonebook_page(2, 5);	cur.execute("SELECT * FROM get_phonebook_page(%s,%s);", (page,size))
Delete by name/phone	PROCEDURE	CALL delete_contact(p_name=>'Alice');	cur.execute("CALL delete_contact(p_name => %s);", (name,))